```
    }
}
```

❶ Injects an HttpClient using DI instead of an IHttpClientFactory

❷ The GetLatestRates() logic encapsulates the logic for interacting with the API.

❸ Uses the HttpClient the same way as before

We can then inject this `ExchangeRatesClient` into consuming services, and they don't need to know anything about how to make HTTP requests to the remote service; they need only to interact with the typed client. We can update listing 33.3 to use the typed client as shown in the following listing, at which point the API endpoint method becomes trivial.

Listing 33.6 Consuming a typed client to encapsulate calls to a remote HTTP server

```
app.MapGet("/", async (ExchangeRatesClient ratesClient) =>  ❶
    await ratesClient.GetLatestRates());                    ❷
```

❶ Injects the typed client using DI

❷ Calls the typed client's API. The typed client handles making the correct HTTP requests.

You may be a little confused at this point. I haven't mentioned how `IHttpClientFactory` is involved yet!

The `ExchangeRatesClient` takes an `HttpClient` in its constructor. `IHttpClientFactory` is responsible for creating the `HttpClient`, configuring it to call the remote service and injecting it into a new instance of the typed client.

You can register the `ExchangeRatesClient` as a typed client and configure the `HttpClient` that is injected in `ConfigureServices`, as shown in the following listing. This is similar to configuring a named client, so you can register additional